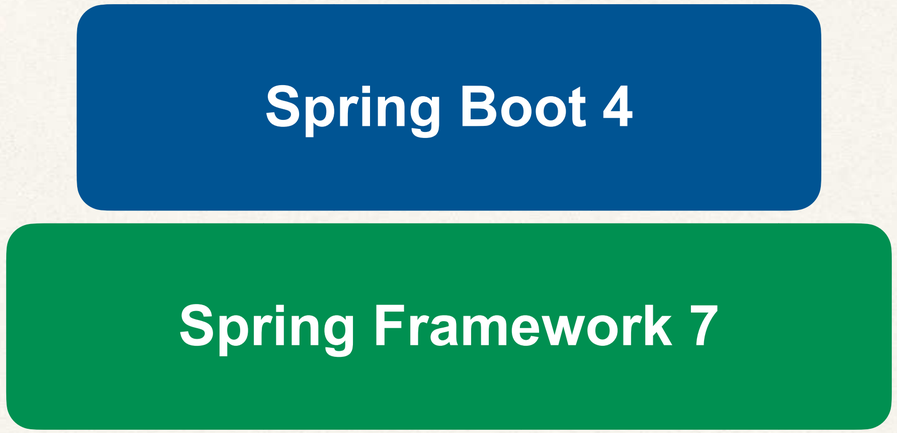
What’s New in Spring Boot 4

© luv2code LLC

Spring Boot 4

- Released in November 2025
- Spring Boot 4 is based on Spring Framework 7



© luv2code LLC

Key Themes

- Faster
  - Runs on modern Java with improved startup time and lower memory usage
- Leaner
  - A new modular design reduces classpath bloat
- Safer
  - Null safety support and updated security to reduce runtime risks
- Enhanced Cloud Readiness
  - Default Kubernetes probes out of the box and enhanced observability support

© luv2code LLC

Categories of Changes

- Modernization & Ecosystem Alignment
- Developer Productivity
- Runtime & Performance
- Cloud Readiness & Observability

© luv2code LLC

Breaking Changes Reminder - History

- In 2023, **Spring Boot 3** introduced major breaking changes
  - Package name changes: from `javax.*` to `jakarta.*`
- Migrating from Spring Boot 2 to Spring Boot 3 was A LOT OF WORK
- Many projects delayed the migrating to Spring Boot 3 due to level of effort

© luv2code LLC

Migrating to Spring Boot 4 is MUCH easier

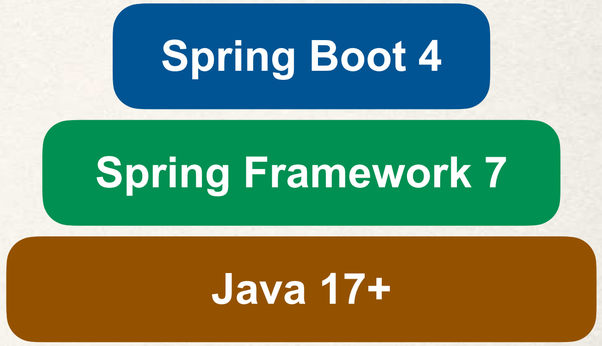
- This is an incremental upgrade with very few breaking changes
- Your existing Spring Boot 3 knowledge still applies: DI, IoC, REST APIs

- Our course code still works … minor changes needed
  - pom.xml version: `<version>4.0.0</version>`
  - Minor updates for AOP Maven dependency and Jackson 3 packages

© luv2code LLC

Spring Boot 4 - Incremental Upgrade

- Modern upgrade across the entire Spring ecosystem
- Builds on Spring Framework 7
- Java 17+ as the baseline (Java 21/25 recommended)
- Ecosystem upgrades: Hibernate 7, JUnit 6, Jackson 3, Tomcat 11 …



© luv2code LLC

Key Features

First-Class API Versioning

Modern HTTP Clients

Jackson 3

Modularization

Null-Safety with JSpecify

© luv2code LLC

Key Feature #1: First-Class API Versioning

- Version APIs via path, header, query, or media type
- No more custom versioning logic or filters
- Consistent versioning strategy across teams
- A clean, built-in solution that makes API versioning simple

First-Class API Versioning

© luv2code LLC

Key Feature #2: Jackson 3

Jackson 3

- Jackson 3 is now the default JSON library for Spring Boot 4
- Package rename
  - `com.fasterxml.jackson` renamed to `tools.jackson`
  - Affects code only if you manually use Jackson classes
- `JsonMapper` is Jackson 3's preferred API
  - Spring Boot 4 auto-configures a `JsonMapper` bean
- Impact on our course
  - Only our REST API PATCH example needs a small update … due to manually using Jackson classes
  - Normal REST JSON handling works the same

© luv2code LLC

Key Feature #3: Null-Safety with JSpecify

- What is JSpecify?
  - A separate framework that provides null-safety annotations: @Nullable, @NonNull, …
  - Makes null intent explicit … IDEs catch null issues before runtime
- What's new in Spring Boot 4
  - Spring APIs now annotated with JSpecify *(used internally in the framework)*
  - JSpecify works across all Spring APIs: controllers, services, repositories, DTOs, …
- JSpecify is optional to use … not a breaking change

Null-Safety with JSpecify

© luv2code LLC

Key Feature #4: Modern HTTP Clients

- RestClient API
  - Cleaner, fluent replacement for RestTemplate (future direction)
- Interface Clients with @HttpExchange
  - Write a simple Java interface then Spring generates the client for you
- Benefits
  - Less boilerplate
  - Easier testing
  - Clearer code than RestTemplate
  - More consistent than writing manual WebClient or RestTemplate calls

Modern HTTP Clients

© luv2code LLC

Key Feature #5: Modularization

Modularization

- What Changed
  - The large `spring-boot-autoconfigure` JAR is now split into 70+ focused modules
  - Starters now pair with matching test starters (e.g., `webmvc` + `webmvc-test`)
- Benefits
  - Loads only what your app needs
  - Faster startup, smaller footprint
- Developer Impact
  - If you are using starters then no changes needed
  - Custom dependency sets should review module names

© luv2code LLC

Spring Boot 4 Resources

Spring Boot 4 Blog Series - Road to GA

https://spring.io/blog/2025/09/02/road_to_ga_introduction

Spring Boot 4 Release Notes

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Release-Notes

Spring Boot 4 Migration Guide

https://github.com/spring-projects/spring-boot/wiki/Spring-Boot-4.0-Migration-Guide

© luv2code LLC

Course Migration Notes
Spring Boot 4 - AOP

© luv2code LLC

Migrating Course Code to Spring Boot 4

- This is an incremental upgrade with very few breaking changes
- Most of our course code just works
- Minor changes needed
  - pom.xml version: `<version>4.0.0</version>`
  - Minor updates for AOP Maven dependency and Jackson 3 packages

© luv2code LLC

AOP in Spring Boot 4

- Starting with Spring Boot 4, the AOP starter was renamed
- Update the pom.xml,
  - Old: `spring-boot-starter-aop`
  - New: `spring-boot-starter-aspectj`

No Java code changes required

Only update the pom.xml

Old

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

New - Spring Boot 4

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```


© luv2code LLC

Development Process

Step-By-Step

1. Start with Spring Boot 3 project that uses AOP
2. Perform the migration to Spring Boot 4: update pom.xml
3. Verify successful migration

Course Migration Notes
Spring Boot 4 - Jackson 3

© luv2code LLC

Spring Boot 4 and Jackson 3

- Jackson 3 is now the default JSON library for Spring Boot 4
- Package rename
  - `com.fasterxml.jackson` renamed to `tools.jackson`
  - Affects code only if you manually use Jackson classes
- `JsonMapper` is Jackson 3's preferred API
  - Spring Boot 4 auto-configures a `JsonMapper` bean
- Impact on our course
  - Only our REST API PATCH example needs a small update … due to manually using Jackson classes
  - Normal REST JSON handling works the same

© luv2code LLC

Development Process

Step-By-Step

1. Perform the migration to Spring Boot 4: update pom.xml
2. In pom.xml, update web starter (replace deprecated)
3. Update Jackson 3 package imports
4. Inject JsonMapper
5. Update PATCH coding accordingly

Step 1: Update pom.xml

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.x.y</version>
  <relativePath/>
</parent>
```

Use the current version of Spring Boot 4.0.0, 4.3.5, 4.7.9 etc …

© luv2code LLC

Step 2: Update web starter

Old

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Deprecated starter name

New - Spring Boot 4

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

New starter name

© luv2code LLC

Step 3: Update Jackson 3 package imports

Old

```
import com.fasterxml.jackson.databind.ObjectMapper;
```

New - Jackson 3 - Spring Boot 4

```
import tools.jackson.databind.json.JsonMapper;
```


© luv2code LLC

Step 4: Inject JsonMapper

- `JsonMapper` is Jackson 3's preferred API
  - Spring Boot 4 auto-configures a `JsonMapper` bean

```
@RestController
@RequestMapping("/api")
public class EmployeeRestController {

    private EmployeeService employeeService;

    private JsonMapper jsonMapper;

    @Autowired
    public EmployeeRestController(EmployeeService theEmployeeService, JsonMapper theJsonMapper) {
        employeeService = theEmployeeService;
        jsonMapper = theJsonMapper;
    }

…
```


© luv2code LLC

Step 5: Update PATCH coding accordingly

```
@PatchMapping("/employees/{employeeId}")
public Employee patchEmployee(@PathVariable int employeeId,
                              @RequestBody Map<String, Object> patchPayload) {

    Employee tempEmployee = employeeService.findById(employeeId);

    ...

    Employee patchedEmployee = jsonMapper.updateValue(tempEmployee, patchPayload);

    Employee dbEmployee = employeeService.save(patchedEmployee);

    return dbEmployee;
}
```


© luv2code LLC

Spring Boot 4 - REST API Versioning

© luv2code LLC

REST API Versioning

- REST APIs evolve over time
- Existing REST clients cannot always upgrade immediately (external etc)
- Spring Boot 4 provides built-in API versioning support
- Goal: support change without breaking existing clients

© luv2code LLC

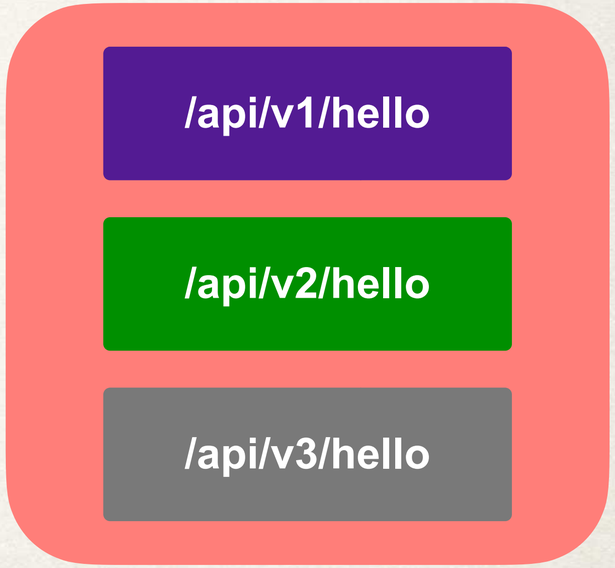
The Problem

- New features often require breaking changes
- Removing or changing fields can break existing clients
- Multiple client versions may exist at the same time (phased approach)
- We need an easy way to run multiple API versions side by side

© luv2code LLC

The Use Case

- Older clients continue using v1 of REST API
- New clients move to v2 or v3
- Expose the same REST endpoint but provide different behavior
- Backend selects the correct version automatically



© luv2code LLC

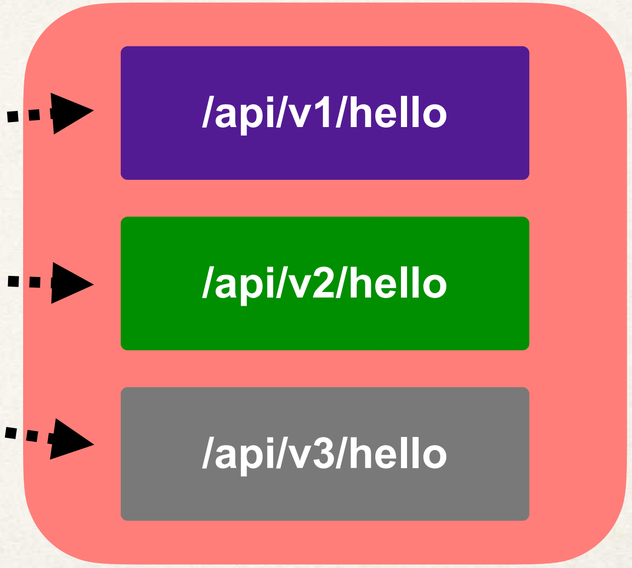
Hello World REST API

- One logical endpoint: /hello
- Three API versions … version comes from the request path
- No duplicate controllers or manual coding

```
GET http://localhost:8080/api/v1/hello

GET http://localhost:8080/api/v2/hello

GET http://localhost:8080/api/v3/hello
```



© luv2code LLC

Development Process

Step-By-Step

1. Develop REST controller with version support

2. Configure Path Segment versioning in application.properties

Step 1: Develop REST controller with version support

```
@RestController
public class HelloWorldRestController {

    @GetMapping(path="/api/{version}/hello", version = "1")
    public String helloV1() {
        return "Hello World from API v1";
    }

    @GetMapping(path="/api/{version}/hello", version = "2")
    public String helloV2() {
        return "Howdy World from API v2";
    }

    @GetMapping(path="/api/{version}/hello", version = "3")
    public String helloV3() {
        return "Hey Hey World from API v3";
    }
}
```

GET http://localhost:8080/api/v1/hello

GET http://localhost:8080/api/v2/hello

GET http://localhost:8080/api/v3/hello

© luv2code LLC

Step 2: Configure Path Segment versioning in application.properties

File: application.properties

```
spring.mvc.apiversion.use.path-segment=1
```

Index in the path

0 1 2

```
path="/api/{version}/hello"
```

```
GET http://localhost:8080/api/v1/hello
GET http://localhost:8080/api/v2/hello
GET http://localhost:8080/api/v3/hello
```


© luv2code LLC

A bit more on placeholder

```
GET http://localhost:8080/api/v1/hello
GET http://localhost:8080/api/v2/hello
GET http://localhost:8080/api/v3/hello
```

```
path="/api/{version}/hello"
```

The placeholder is in curly braces

Placeholder can have any name
Does not have to be named “version”

could also use “ver”, “apiVersion”, or anything else

© luv2code LLC

How Routing Works

```
GET http://localhost:8080/api/v1/hello
GET http://localhost:8080/api/v2/hello
GET http://localhost:8080/api/v3/hello
```

- Spring extracts the version from URL
- Version is parsed automagically (strips leading non-numeric … only keeps numeric)
- Spring matches the request to the correct request mapping

```
@RestController
public class HelloWorldRestController {

    @GetMapping(path="/api/{version}/hello", version = "1")
    public String helloV1() {
        return "Hello World from API v1";
    }

    @GetMapping(path="/api/{version}/hello", version = "2")
    public String helloV2() {
        return "Howdy World from API v2";
    }

    @GetMapping(path="/api/{version}/hello", version = "3")
    public String helloV3() {
        return "Hey Hey World from API v3";
    }
}
```

GET http://localhost:8080/api/v1/hello

GET http://localhost:8080/api/v2/hello

GET http://localhost:8080/api/v3/hello

© luv2code LLC

More on parsing version from URL

- Version is parsed automagically strips leading non-numeric … only keeps numeric
- Spring looks for the number in the path segment
- Ignores any leading letters or symbols before the number

```
GET http://localhost:8080/api/v1/hello

GET http://localhost:8080/api/v2/hello

GET http://localhost:8080/api/v3/hello
```

```
GET http://localhost:8080/api/abc1/hello

GET http://localhost:8080/api/xyz2/hello

GET http://localhost:8080/api/a-z3/hello
```

Avoid using URL reserved characters `:/?#[]@!$&'()*+,;=`

© luv2code LLC

Benefits of API Versioning

- Declarative API versioning
- Clean controller code (no special logic, no conditionals)
- Easy to add new versions
- Minimal impact on existing clients

© luv2code LLC